

4.1.2 Back-End

4.1.2.1 Overview

The backend of the **Capital Taxi** system is built using Node.js, a powerful and efficient JavaScript runtime environment. Node.js is known for its non-blocking, event-driven architecture, making it ideal for building scalable and high-performance applications. The backend handles requests from the mobile clients (riders and drivers), processes application logic, interacts with the MongoDB database, and sends appropriate responses back to the client.

4.1.2.2 Technology Stack

- **Node.js:** The primary runtime environment used to execute JavaScript on the server side.
- **Express.js:** A minimalist web application framework for Node.js, used to create RESTful API endpoints and handle HTTP requests.
- **MongoDB:** A NoSQL document-based database used to store and manage application data.
- **Mongoose:** An ODM (Object Data Modeling) library for MongoDB and Node.js, used to define schemas and perform structured interactions with the database.
- **JWT (JSON Web Tokens):** Used for secure user authentication and authorization between clients and the server.



Node.js is a powerful, open-source, cross-platform JavaScript runtime built on Google Chrome's V8 JavaScript engine. It enables developers to use JavaScript for backend (server-side) development, producing dynamic responses for client applications.

Why Node.js?

Node.js was chosen for this project for several reasons:

1. Asynchronous and Event-Driven

Node.js uses non-blocking architecture, which allows it to handle multiple simultaneous client requests efficiently — essential for real-time services like trip requests, driver responses, and notifications.

2. Single Programming Language

Using JavaScript for both front-end (mobile client) and back-end improves productivity and simplifies the development workflow.

3. High Performance

Node.js compiles JavaScript into native machine code using the V8 engine, providing fast performance for backend operations.

4. Rich Ecosystem

Its ecosystem (npm) provides access to thousands of libraries, making development faster and more flexible.

5. Scalability

Node.js is ideal for scalable network applications such as Capital Taxi, which requires handling many concurrent trip requests and driver updates.

6. Community Support

Node.js has strong community support, rich documentation, and plenty of third-party modules.

Key Features of Node.js

- Non-Blocking I/O: Lightweight and efficient when handling real-time, data-intensive features like trip updates and notifications.
- Single-Threaded Event Loop: Efficient resource management when processing concurrent user requests.
- Event Loop: Handles asynchronous operations and callbacks, crucial for real-time interactions between drivers and passengers.
- npm (Node Package Manager): Access to a vast library of reusable packages, enabling faster and modular development

Node.js in the Capital Taxi System

In **Capital Taxi**, the backend is responsible for coordinating the entire ride-hailing process. Some of the major features handled by Node.js include:

- **User & Driver Authentication**

Secured via JWT, allowing login and token-based session management.

- **Trip Management**

Creating trip requests, assigning drivers, updating trip statuses, and ending trips.

- **Driver Discovery & Assignment**

Finding the nearest available drivers using location data, then assigning them to trip requests.

- **Real-Time Communication**

Supporting real-time updates through Socket.IO or REST endpoints for notifications, driver arrivals, and trip status.

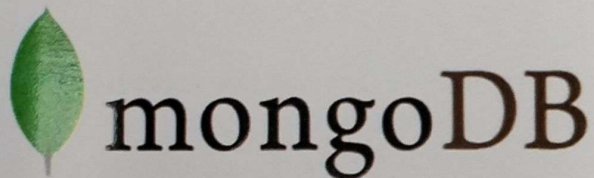
- **Payments**

Handling both cash and card payment processing.

- **Admin Monitoring**

APIs for the admin dashboard to view current trips, monitor driver activity, and manage users.

MongoDB



MongoDB is used as a database to store and manage structured data in a flexible and scalable way. As a NoSQL, document-oriented database, MongoDB is highly suitable for dynamic applications like Capital Taxi, where data models for users, drivers, trips, and ratings may evolve.

Why MongoDB?

- Flexible Schema: Allows quick updates to models like Trip, User, or Driver without strict constraints.
- High Availability: Supports replica sets for fault tolerance.
- Scalability: Easily handles growing data volume (e.g., ride history, transactions, driver ratings).
- Document-Based Storage: Each record (document) is stored in JSON-like format making it easy to work with in JavaScript-based environments.

MongoDB in Capital Taxi

In Capital Taxi, MongoDB is used to manage collections such as:

- users – storing passenger information.
- drivers – managing driver profiles, availability, and location.
- trips – recording trip details including status, distance, and fare.
- payments – tracking payment methods and transaction records.
- ratings – collecting user feedback after each trip.

Each collection is modeled using Mongoose schemas to enforce data structure and validation rules.

Overview

Technology Stack "Node.js, Express, MongoDB, mongoose and JWT"

2.4.8 Designing User-Friendly Interfaces	17
2.4.9 Maintaining Backend Scalability.....	17
2.4.10 Ensuring Compliance with Legal and Regulatory Requirements.....	17
3. Chapter Three.....	18
3.1 Requirements & Analysis.....	19
3.1.1 Functional Requirement.....	19
3.1.2 Non-Functional Requirement.....	20
3.1.3 Hardware requirements.....	22
3.1.4 Software requirements.....	23
3.2 UML Diagrams.....	25
3.2.1 Use Case Diagram.....	25
3.2.2 Sequence Diagram-Request Ride.....	26
3.2.3 Sequence Diagram-Register & Verify Driver.....	27
3.2.4 Activity Diagram- Trip Lifecycle.....	28
3.2.5 Activity Diagram-Complaint Handling.....	29
3.2.6 Class Diagram.....	30
3.2.7 Flow Chart.....	31
3.2.8 Component Diagram.....	32
3.2.9 Deployment Diagram (Infrastructure).....	33
Chapter Four.....	34

4.3.4 ERD

